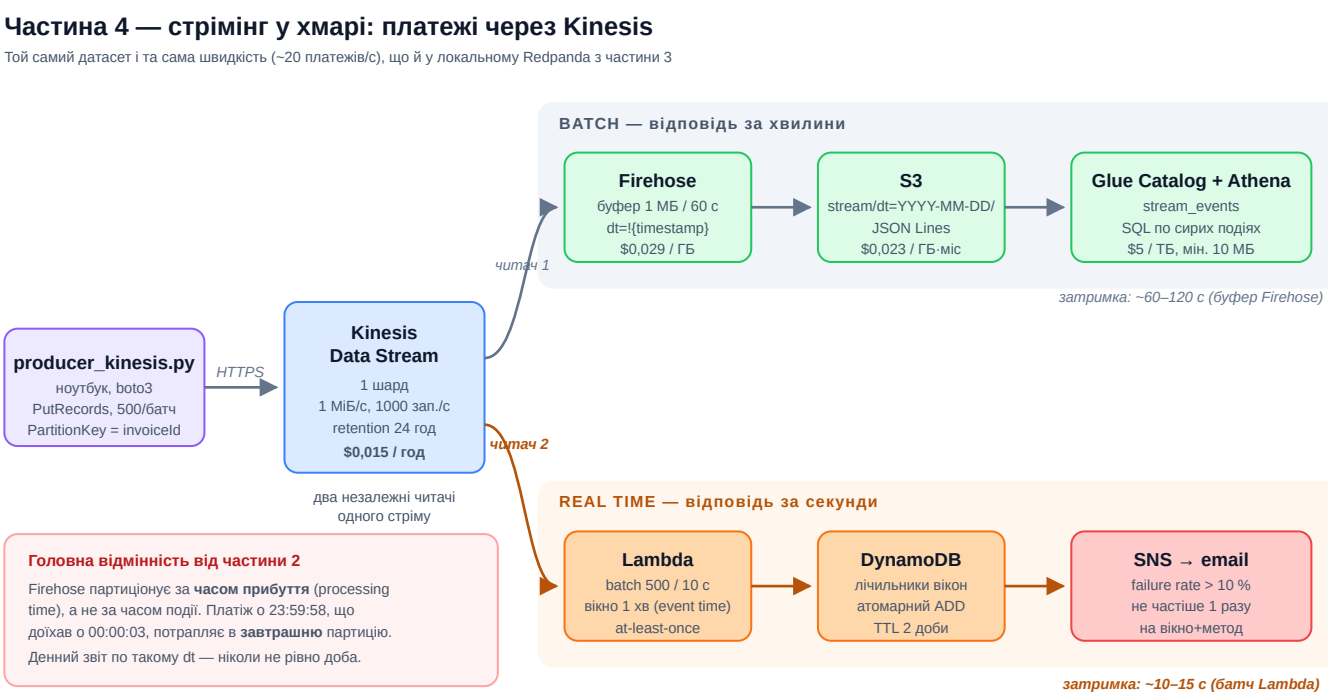
Частина 4: стрімінг у хмарі (Kinesis)

OOAD, лабораторна 8 · timlim24 · github.com/timlim24/OOAD-lab-8

part4-cloud-streaming/README.md

Частина 4 — Стрімінг у хмарі: Kinesis → Firehose → S3 → Athena + алерт через SNS

Діаграма




```
flowchart LR
    P["producer_kinesis.py<br/>ноутбук · boto3<br/>PutRecords, 500/батч<br/>PartitionKey = invoiceId"]
    K["Kinesis Data Stream<br/>1 шард · 1 МіБ/с<br/>retention 24 год<br/>$0,015/год"]

    subgraph BATCH["BATCH — хвилини"]
        F["Firehose<br/>буфер 1 МБ / 60 с<br/>dt=!{timestamp}"]
        S["S3<br/>stream/dt=YYYY-MM-DD/<br/>JSON Lines"]
        A["Glue Catalog + Athena<br/>stream_events"]
    end

    subgraph RT["REAL TIME — секунди"]
        L["Lambda<br/>batch 500 / 10 с<br/>вікно 1 хв (event time)"]
        D["DynamoDB<br/>лічильники вікон<br/>атомарний ADD · TTL"]
        N["SNS → email<br/>failure rate > 10%"]
    end

    P -->|HTTPS| K
    K -->|читач 1| F --> S --> A
    K -->|читач 2| L --> D --> N
```

Звідки беруться дані. Той самий датасет платежів, що й у частинах 2–3: один день (`dt=2026-10-01`, 101 084 події, 33 361 платіж) програється з локального файлу в Kinesis у порядку `receivedAt` із тим самим стисканням часу (~20 платежів/с, speed factor ≈ 52×). Продукт не міняється навмисно — щоб локальний і хмарний пайплайни були порівнянними по тих самих числах.

Запуск

```
aws login
cd scripts
./setup.sh you@example.com     # ~3 хв: створює все й друкує наступні кроки
# підтвердити підписку SNS у пошті — без цього алерти тихо нікуди не йдуть
# ./setup.sh без аргументу — топік без підписки, алерти видно в логах Lambda

python ../producer_kinesis.py \
    --file ../../part3-streaming-local/data/events-2026-10-01.jsonl \
    --stream ooad-lab8-payments --rate 200 --fail-rate 0.30

aws logs tail /aws/lambda/ooad-lab8-alerting --follow      # дивитися алерти
aws s3 ls s3://<bucket>/stream/ --recursive | head         # за ~2 хв

# Athena: sql/stream_table.sql
./teardown.sh                  # ОБОВ'ЯЗКОВО
```

Файл	Що це
producer_kinesis.py	реплей у Kinesis через `PutRecords`
lambda/alert.py	віконна частка відмов → SNS
scripts/setup.sh	усе: стрім, Firehose, Lambda, DynamoDB, SNS, IAM
scripts/teardown.sh	видалити все (шард коштує грошей)
sql/stream_table.sql	таблиця Athena над даними зі стріму

> Результати запитів Athena → results/athena_stream.md

Дві пастки, на які варто чекати

`pip install boto3` недостатньо. Якщо машина автентифікується через `aws login`, botocore відмовляється читати ці креденшали без `botocore[crt]` і падає з `MissingDependencyException` — хоча сам `aws` CLI у тому ж терміналі працює. Тому в requirements.txt стоїть саме `botocore[crt]`.

Копіювання публічного бакета теж має пастку (частина 2): `aws s3 sync` за замовчуванням зберігає теги, тобто викликає `GetObjectTagging` на **джерелі**. Публічний бакет дає `s3:GetObject`, але не `GetObjectTagging`, і всі 31 копія падають з `AccessDenied`, хоча дані читаються чудово. Лікує `--copy-props none`.

Чим хмарний пайплайн відрізняється від локального Redpanda

1. Один стрім — два незалежні читачі

Локально консюмер один, і якби знадобився другий — я б підняв другу групу. У Kinesis Firehose і Lambda читають **той самий стрім незалежно**, кожен зі своєю позицією. Це та сама ідея, що consumer groups у Kafka, тільки вона зашита в сервіс: Firehose — це не код, який я написав, а керована підписка.

Ключова архітектурна річ, яку це дає: **batch і real-time читають одні й ті самі події**. Не дві копії, не два записи з аплікації — один лог, два споживачі з різними дедлайнами. Саме це видно на діаграмі частини 1.

2. Партиціонування за часом прибуття, а не за часом події

Це найважливіша відмінність від частини 2. Firehose підставляє в префікс `!{timestamp:yyyy-MM-dd}` **приблизний час прибуття запису в стрім** — processing time. У частині 2 `dt` був **днем події** (перевірено: 0 подій, де день `modifiedDate` ≠ `dt`).

Наслідок: платіж о 23:59:58, що доїхав о 00:00:03, лягає в **завтрашню** партицію. Денний звіт по такому `dt` — ніколи не рівно доба бізнесу.

Полагодити можна двома способами, і обидва — це частина 1 у мініатюрі: - **dynamic partitioning** Firehose із JQ-виразом по `modifiedDate` (+$0,02/ГБ); - **нічний Glue job**, який перепартиціонує за часом події, — тобто batch-дорога існує саме для того, щоб прибирати за streaming-дорогою.

3. Часткові відмови — норма, а не виняток

`PutRecords` повертає **HTTP 200** і при цьому поле `FailedRecordCount`. Задроселений запис — це поле в успішній відповіді, не виняток. Код, який ловить тільки exceptions, **мовчки втрачає дані**.

Тому в producer_kinesis.py є цикл, що перевідправляє **тільки відхилені** записи з експоненційним backoff. Перевідправка всього батчу продублювала б ті, що пройшли.

У Kafka-клієнта цього немає в коді, бо librdkafka/kafka-python ховають усе це за власною чергою і ретраями. Різниця не в надійності, а в тому, **хто пише цей код** — ви чи бібліотека.

4. Lambda не має стану, і це видно

Локальний консюмер тримає вікна, watermark і множину побачених подій у пам'яті процесу. Lambda бачить **один батч і помирає**. Хвилинне вікно event time не влазить у батч: події однієї хвилини приходять у кількох батчах, кількох інвокаціях і — через паралельне читання шардів — у кількох одночасних виконаннях.

Тому лічильники винесені в **DynamoDB з атомарним `ADD`**: операція комутативна, тож паралельні інвокації оновлюють те саме вікно без локів і без втрати лічби. Повторний алерт на те саме вікно відсікається умовним записом (`attribute_not_exists(alerted)`).

Чого Lambda-версія не вміє, а локальний консюмер уміє:

	Redpanda + consumer.py	**Kinesis + Lambda**
watermark	є, `max(modifiedDate)`	**немає** — це властивість усього стріму, а Lambda бачить фрагмент
відкидання запізнілих	є, з `--grace`	**немає** — запис у закрите вікно просто додається
гарантія	at-least-once + дедуплікація в пам’яті	at-least-once, **дедуплікації немає взагалі**
стан при рестарті	втрачається (тільки офсети)	у DynamoDB, переживає все
ціна стану	0	round trip на вікно на батч

Ці три прогалини — не баги, які треба полагодити в `alert.py`. Це **рівно те, за що платять у Managed Flink**: keyed state у пам’яті поруч із даними, чекпоінти стану разом з офсетами стріму, справжні watermark-и. Коли на захисті питають «а чому не зробити все на Lambda» — ось відповідь у трьох рядках.

5. Затримка

	локально	**у хмарі**
продюсер → брокер	мілісекунди (`linger_ms=50`)	~20–50 мс (HTTPS)
брокер → алерт	мілісекунди	**~10–15 с** (batching window Lambda)
брокер → дані в S3	—	**~60–120 с** (буфер Firehose)
дані доступні в SQL	—	`+ MSCK REPAIR`

Локально алерт з’являється практично миттєво, бо консюмер — це процес у сусідньому терміналі. У хмарі мінімум **10 секунд** додає Lambda batching window (його можна поставити в 0 — ціною інвокації на кожен запис) і **до хвилини** — буфер Firehose, у якого 60 секунд це *мінімум*, нижче не ставиться.

Хмара не швидша. Вона масштабованіша. Redpanda на ноутбуці обробить ці 60 подій/с із меншою затримкою, ніж уся ця конструкція. Різниця починається на 60 000 подій/с, коли ноутбук закінчується, а Kinesis просто додає шарди.

6. Операційна різниця

	Redpanda локально	**Kinesis у хмарі**
підняти	`docker compose up` — 30 с	`setup.sh` — ~3 хв, 2 IAM-ролі, 7 ресурсів
що треба знати	Kafka API	IAM, ARN-и, trust policy, event source mapping
масштабування	`--smp`, партиції вручну	`update-shard-count`, автоматично
коли падає	видно в терміналі	CloudWatch Logs, три різні лог-групи
ціна помилки	нуль	шард цокає $0,015/год, поки не видалиш
забути вимкнути	нічого не станеться	**$11/міс**

Половина setup.sh — це IAM. Це чесна ціна керованих сервісів: вони не треба адмініструвати, але треба **описувати права доступу між ними**.

Скільки це коштувало

Метрики — з CloudWatch, прайс-лист us-east-1. Реплей: 101 084 записи, 43,9 МБ, середній запис **433 байти**.

Сервіс	**Тариф**	**Обсяг**	**$**
Kinesis shard-hour	$0,015 / шард·год	1 год	**0,0150**
Firehose ingestion	$0,029 / ГБ	**0,518 ГБ** (після округлення)	**0,0150**
DynamoDB on-demand write	$1,25 / млн WCU	6 500	**0,0081**
Kinesis PUT payload units	$0,014 / млн	101 084	0,0014
CloudWatch Logs	$0,50 / ГБ	≈ 3 МБ	0,0015
Lambda	$0,0000166667 / ГБ·с	205 × 1,2 с × 0,25 ГБ	0,0010
S3 (PUT + storage)		41 файл, 42 МБ	0,0013
Athena	$5 / ТБ, мін. 10 МБ	≈ 10 запитів	0,0005
SNS	перші 1 млн безкоштовно	635 публікацій	0
		Разом	**≈ $0,044**

Три цифри, які варто пояснити

1. Firehose: 43,9 МБ даних, платимо за 518 МБ. Firehose округлює **кожен запис до 5 КБ**. Наші — 433 байти, тобто `5120 / 433 = 11,8×` переплати: `101 084 × 5 КБ = 518 МБ` замість 43,9 МБ. Якби продюсер пакував по 10 платежів в один запис (~4,3 КБ), ця стаття впала б майже вдесятеро.

2. DynamoDB дорожчий за Lambda у 8 разів — $0,0081 проти $0,0010. **Зберігання стану дорожче за обчислення над ним.** Це прямий аргумент за Flink: там ті самі лічильники живуть у keyed state у пам'яті воркера й чекпоінтяться пачкою, а наш Lambda робить ~32 `UpdateItem` на інвокацію — окремий мережевий похід на кожну комірку вікно×метод.

3. Шард коштує однаково, чи тече через нього щось. $0,015/год — це $10,95/місяць за шард, який нічого не робить. Redpanda в Docker за той самий місяць — $0.

Троттлінг: один шард виявився замалим

Два прогони того самого дня, різниця лише в `--rate`:

	`--rate 150`	`--rate 200`
sent	101 084	101 084
throttled	**0**	**69 162**
DROPPED	0	0
фактичний темп	**149,8 платежів/с**	**154,1** (замовлено 200)

Один шард тримає 1 000 записів/с або 1 МіБ/с. При 150 платежах/с це 450 подій/с — із запасом; при 200 середнє вже 600 подій/с, а вечірній пік утричі щільніший за середнє, і стеля пробивається. **Визначає стелю пік, а не середнє.**

Жоден запис не втрачено: `PutRecords` повертає **HTTP 200** із полем `FailedRecordCount`, і цикл у producer_kinesis.py перевідправляє **тільки відхилені** записи. Ціна троттлінгу — час: 216,5 с замість 167, тобто фактичні 154 платежі/с замість замовлених 200. Kinesis не відмовляє, він пригальмовує, і побачити це можна лише в полях відповіді.

Якщо масштабувати до повного датасету

	наш реплей	3 млн подій/добу
Kinesis / міс	—	**$12,21**
Firehose / міс (по 5 КБ)	—	**$13,05**
Firehose / міс (пакуючи по 10)	—	**$1,36**
DynamoDB / міс	—	≈ $7,30
S3 + Lambda + Athena / міс	—	≈ $3
Разом	**$0,044**	**≈ $36/міс** (або $24 з пакуванням)

Еквівалентний self-hosted Kafka — мінімум три `t3.medium` у різних AZ (≈ $90/міс) плюс людина, яка їх обслуговує. **Перелом не в ціні заліза, а в ціні години інженера.**

Повний розбір: results/cost.md.

Ресурси видалено

`teardown.sh` виконано — Kinesis, Firehose, Lambda, DynamoDB, SNS і обидві IAM-ролі; перевірено порожніми списками. Свідомо лишені S3 (1,4 ГБ, **$0,033/міс**) і Glue-каталог (free tier), щоб запити Athena лишалися відтворюваними.

Що порівняти з частиною 3 — і що з цього вийшло

Запит 2 у sql/stream_table.sql рахує ту саму частку відмов по `paymentMethod`, що й локальний консюмер. Фактичний результат:

paymentMethod	terminal (стрім)	failures	%	terminal (локально, ч. 3)
apple_pay	8 412	2 896	34,43	**8 412**
qr	700	241	34,43	**700**
card	18 406	6 336	34,42	**18 406**
google_pay	5 843	1 991	34,07	**5 843**

Кількість термінальних подій збігається до одиниці. Батч над даними зі стріму і стрім-консюмер рахують одне й те саме — відрізняється лише дедлайн, за яким вони дають відповідь.

Частка 34,1–34,4 % — рівно те, що й має бути при `--fail-rate 0.30`: `0,30 + 0,70 × 0,0602 = 34,2 %`.

Наскрізний облік теж зійшовся: надіслано 101 084 записи → Athena бачить **101 084 події, 33 361 платіж, рівно 1 001 дублікат** — стільки ж дублікатів, скільки їх фізично у вихідному файлі.

Що могло б не зійтися

Якби Athena показала менше, причин було б рівно три, і варто вміти їх розрізняти:

1. **продюсера зупинили раніше, ніж він дослав усе** — видно в його ж підсумковому рядку `sent`;
2. **останній буфер Firehose не встиг флашнутися** — саме це й сталося в процесі: перший запит показав 99 785 подій із `last_event` 23:02, і лише через кілька хвилин з'явилися решта 1 299. Неповний буфер чекає на 60-секундний інтервал, бо не добирає до 1 МБ. **Це і є «eventual» у eventual consistency: дані не загублені, вони ще в буфері**;
3. **записи, які `PutRecords` відхилив, а продюсер здався ретраїти** — рядок `DROPPED` у підсумку. Єдина з трьох причин, що є справжньою втратою даних, і в локальному Redpanda її не буває взагалі.

Athena над даними, що прийшли через стрім

Пайплайн: `producer_kinesis.py` → Kinesis (1 шард) → Firehose → S3 → Glue → Athena. Реплей: `--rate 150 --fail-rate 0.30`, день `dt=2026-10-01`.

Продюсер:

```
sent      : 101,084 records for 33,361 payments
throttled : 0 (retried)
forced to failure: 9,576 payments
wall time : 222.7s (149.8 payments/s)
```

Запит 1 — чи все доїхало

```
SELECT count(*) AS events, count(DISTINCT invoiceId) AS payments,
       count(*) - (SELECT count(*) FROM (SELECT DISTINCT * FROM lab8.stream_events))
         AS duplicate_rows,
       min(modifiedDate) AS first_event, max(modifiedDate) AS last_event
FROM lab8.stream_events;
```

events	payments	duplicate_rows	first_event	last_event
101 084	**33 361**	**1 001**	2026-10-01T00:00:01Z	2026-10-01T23:59:56Z

Облік зійшовся повністю. Продюсер надіслав 101 084 записи — Athena бачить 101 084. Firehose доставив рівно стільки, скільки Kinesis прийняв (`DeliveryToS3.Records` = `IncomingRecords`). Нуль втрат на всьому шляху ноутбук → Kinesis → Firehose → S3.

1 001 дублікат — рівно стільки, скільки їх фізично у вихідному файлі (`count(*) - count(DISTINCT *)` по `day01.jsonl`). Це наскрізний доказ, що виправлення детермінованої причини відмови працює: через чотири сервіси структура дублікатів датасету збереглася байт-у-байт.

> Для скриншота: Athena console → Query results. `QueryExecutionId` кожного запуску — в benchmark.csv.

Запит 2 — частка відмов, звірка з частиною 3

```
WITH dedup AS (
  SELECT DISTINCT invoiceId, status, modifiedDate, paymentMethod
  FROM lab8.stream_events
)
SELECT paymentMethod,
       count_if(status IN ('success','failure')) AS terminal,
       count_if(status = 'failure')              AS failures,
       round(100.0 * count_if(status='failure')
           / nullif(count_if(status IN ('success','failure')),0), 2) AS failure_rate_pct
FROM dedup GROUP BY paymentMethod ORDER BY failure_rate_pct DESC;
```

paymentMethod	terminal	failures	% (Athena, стрім)	terminal (ч. 3, локально)
apple_pay	8 412	2 896	34,43	8 412
qr	700	241	34,43	700
card	18 406	6 336	34,42	18 406
google_pay	5 843	1 991	34,07	5 843

Кількість термінальних подій збігається з локальним підрахунком по тому самому дню до одиниці. Батч над даними зі стріму і стрім-консюмер з частини 3 рахують одне й те саме.

Частка 34,1–34,4 % — рівно те, що й має бути: 30 % примусових відмов плюс природні 6,02 % на решті платежів, тобто `0,30 + 0,70 × 0,0602 = 34,2 %`.

Запит 3 — партиція це processing time, а не event time

```
SELECT dt AS partition_dt,
       min(substr(modifiedDate,1,10)) AS earliest_event_day,
       max(substr(modifiedDate,1,10)) AS latest_event_day,
       count(*) AS events
FROM lab8.stream_events GROUP BY dt ORDER BY dt;
```

partition_dt	earliest_event_day	latest_event_day	events
2026-11-03	2026-10-01	2026-10-01	101 084

Одна партиція, названа днем запуску реплею, містить цілий інший день бізнесу. У частині 2 ті самі події лежали у 31 партиції, названих днями, коли вони сталися (перевірено: 0 подій, де день `modifiedDate` ≠ `dt`).

Причина — у префіксі Firehose: `!{timestamp:yyyy-MM-dd}` підставляє **approximate arrival time** запису в стрім. Нічого не губиться, але `WHERE dt = '2026-10-01'` тут поверне нуль рядків, а в частині 2 — правильну відповідь. Денний звіт по такому `dt` ніколи не дорівнює добі бізнесу.

Полагодити можна двома способами, і обидва — частина 1 у мініатюрі: dynamic partitioning Firehose із JQ-виразом по `modifiedDate` (+$0,02/ГБ), або нічний Glue job, що перепартиціонує за часом події — тобто batch-дорога існує, щоб прибирати за streaming-дорогою.

Файли, які написав Firehose

```
aws s3 ls s3://<bucket>/stream/ --recursive --summarize
```

файлів	**41**
усього	42 496 073 Б
середній розмір	**1 036 490 Б**

Середній файл — рівно ~1 МБ, тобто спрацював `BufferingHints.SizeInMBs = 1`, а не 60-секундний інтервал: при 450 подіях/с по 433 байти мегабайт набирається за ~5 секунд. Останній, неповний буфер дочекався інтервалу — у Athena він з'явився на кілька хвилин пізніше за решту, і доти запит показував 99 785 подій замість 101 084. **Це і є «eventual» у eventual consistency стріму: дані не загублені, вони ще в буфері.**

41 файл на 100 тисяч подій — це мініатюра **small files problem**. За добу реального навантаження таких файлів були б тисячі, і Athena витрачала б більше часу на відкриття файлів, ніж на читання даних. Лікується тим самим нічним CTAS у Parquet із частини 2.

Алерт через SNS

Lambda рахує хвилинні вікна в DynamoDB і публікує в SNS при частці відмов понад 10 %. Приклад із CloudWatch Logs:

```
{"records": 500, "terminal": 170, "skipped": 0, "cells": 26,
 "alerts": [{"window": 1790847960, "method": "card", "rate": 0.1739, "n": 23},
            {"window": 1790848140, "method": "card", "rate": 0.2,    "n": 20}]}
```

Підтвердження, що алерти реально вийшли назовні (підписки на топік немає, тож доказ — метрика):

```
AWS/SNS NumberOfMessagesPublished = 1 271
```

> Лічильники вікон у DynamoDB накопичувалися між кількома реплеями, тому загальна кількість алертів за годину не є чистим виміром одного прогону. Чистий вимір частоти алертів — у частині 3: 55 на базовому дні проти 326 при 30 % відмов.

Фактична вартість прогону

Метрики — з CloudWatch за годину, протягом якої пройшли два повні реплеї (плюс кілька smoke-прогонів). Нижче перераховано **на один реплей** (101 084 записи).

Що реально пройшло через пайплайн

Метрика	За годину (2 реплеї)	На один реплей
`Kinesis IncomingRecords`	202 789	101 084
`Kinesis IncomingBytes`	87 889 473 Б	≈ 43,9 МБ
середній розмір запису	**433 Б**	
`Firehose DeliveryToS3.Records`	**202 789**	101 084
`Lambda Invocations`	409	≈ 205
`DynamoDB ConsumedWriteCapacityUnits`	12 999	≈ 6 500
`SNS NumberOfMessagesPublished`	1 271	≈ 635

Firehose доставив рівно стільки записів, скільки Kinesis прийняв — 202 789 = 202 789. Нуль втрат на цій ділянці, і це вимірювана величина, а не припущення.

Вартість одного реплею (прайс-лист us-east-1)

Стаття	Тариф	Обсяг	$
Kinesis shard-hour	$0,015 / шард·год	1 год	**0,0150**
Kinesis PUT payload units	$0,014 / млн	101 084	0,0014
Firehose ingestion	$0,029 / ГБ	**0,518 ГБ** (після округлення)	**0,0150**
DynamoDB on-demand write	$1,25 / млн WCU	6 500	**0,0081**
Lambda duration	$0,0000166667 / ГБ·с	205 × 1,2 с × 0,25 ГБ	0,0010
Lambda requests	$0,20 / млн	205	0,00004
S3 PUT	$0,005 / 1 000	≈ 60 файлів	0,0003
S3 storage	$0,023 / ГБ·міс	44 МБ	0,0010
Athena	$5 / ТБ, мін. 10 МБ	≈ 10 запитів	0,0005
CloudWatch Logs	$0,50 / ГБ	≈ 3 МБ	0,0015
SNS	перші 1 млн безкоштовно	635	0
		Разом	**≈ $0,044**

Три цифри, які варто пояснити

1. Firehose: 43,9 МБ даних, платимо за 518 МБ. Firehose округлює **кожен запис до 5 КБ**. Наші — 433 байти, тобто `5120 / 433 = 11,8×` переплати: `101 084 × 5 КБ = 518 МБ` замість 43,9 МБ. Це найбільша стаття разом із шардом, і вона цілком рукотворна: якби продюсер пакував по 10 платежів в один запис (~4,3 КБ), рахунок Firehose упав би майже вдесятеро.

2. DynamoDB коштує дорожче за Lambda у 8 разів. $0,0081 проти $0,0010 — **зберігання стану дорожче за обчислення над ним**. Це прямий аргумент за Flink: там ті самі лічильники живуть у keyed state у пам’яті воркера й чекпоінтяться пачкою, а не записуються по одному запиту на вікно на батч. Наш Lambda робить ~32 `UpdateItem` на інвокацію (по одному на комірку вікно×метод у батчі) — і кожен це окремий мережевий похід.

3. Шард коштує однаково, чи тече через нього щось. $0,015/год — це $0,36/добу, **$10,95/місяць** за шард, який нічого не робить. Для порівняння: Redpanda в Docker за той самий місяць — $0.

Троттлінг: 1 шард виявився замалим

Два прогони того самого дня, різниця лише в `--rate`:

	`--rate 150`	`--rate 200`
sent	101 084	101 084
throttled	**0**	**69 162**
DROPPED	0	0
wall time	222,7 с	216,5 с
фактичний темп	**149,8 платежів/с**	**154,1** (замовлено 200)

Один шард тримає 1 000 записів/с або 1 МіБ/с. При 150 платежах/с це 450 подій/с у середньому — із запасом. При 200 середнє вже 600 подій/с, а на вечірньому піку профіль доби втричі щільніший, і стеля пробивається. Отже **стеля одного шарда для цього профілю лежить між 150 і 200 платежами/с** — і визначає її не середнє навантаження, а пік.

Цифра 69 162 рахує всі відхилення, включно з повторними для одного запису.

Важливо, що **жоден запис не загубився**:

```
sent      : 101,084 records for 33,361 payments
throttled : 69,162 (retried)
wall time : 216.5s (154.1 payments/s)
```

Рядка `DROPPED` немає в жодному прогоні, і Athena бачить рівно 101 084 події. `PutRecords` повертає **HTTP 200** з полем `FailedRecordCount` — задроселений запис це поле в успішній відповіді, не виняток. Цикл у producer_kinesis.py перевідправляє **тільки відхилені** записи з експоненційним backoff; перевідправка всього батчу продублювала б ті, що пройшли. Код, який ловив би лише exceptions, тут мовчки втратив би дві третини даних.

Ціна троттлінгу — час: 216,5 с замість розрахункових 167, тобто **фактичні 154 платежі/с замість замовлених 200**. Kinesis не відмовляє — він пригальмовує, і побачити це можна тільки в полях відповіді. Пайплайн, який не рахує `FailedRecordCount`, виглядатиме здоровим і при цьому тихо відставатиме.

Якщо масштабувати до повного датасету

	наш реплей	3 млн подій/добу
Kinesis шардів	1	1 (35 подій/с)
Kinesis / міс	—	$10,95 + $1,26 = **$12,21**
Firehose / міс (по 5 КБ)	—	450 ГБ × $0,029 = **$13,05**
Firehose / міс (пакуючи по 10)	—	**$1,36**
DynamoDB / міс	—	≈ **$7,30**
S3 + Lambda + Athena / міс	—	≈ $3
Разом	**$0,044**	**≈ $36/міс** (або $24 з пакуванням)

Еквівалентний self-hosted Kafka — мінімум три `t3.medium` у різних AZ (≈ $90/міс) плюс людина, яка їх обслуговує. **Перелом не в ціні заліза, а в ціні години інженера.**

Звірка в Cost Explorer

Білінг відстає до доби. Наступного дня:

```
aws ce get-cost-and-usage \
  --time-period Start=2026-11-03,End=2026-11-04 \
  --granularity DAILY --metrics UnblendedCost \
  --group-by Type=DIMENSION,Key=SERVICE
```

Сервіс	$ (факт)
Amazon Kinesis	
Amazon Data Firehose	
Amazon DynamoDB	
Amazon S3	
AWS Lambda	
Amazon Athena	
AmazonCloudWatch	
Разом	

Чи все видалено

`./teardown.sh` виконано. Перевірка:

```
aws kinesis list-streams --region us-east-1 --query StreamNames --output text
aws firehose list-delivery-streams --region us-east-1 --query DeliveryStreamNames --output text
aws lambda list-functions --query 'Functions[?starts_with(FunctionName,`ooad-lab8`)].FunctionName' --output text
aws dynamodb list-tables --query TableNames --output text
aws sns list-topics --query 'Topics[?contains(TopicArn,`ooad-lab8`)]' --output text
```

Усі п’ять повертають **порожньо**. Жодної shard-години більше не нараховується.

Що лишилося свідомо

Ресурс	Чому лишився	Вартість
S3: `payments/` 1,26 ГБ, `curated/` 88 МБ, `stream/` 42 МБ	щоб запити Athena з частин 2 і 4 відтворювалися	**$0,033/міс**
Glue Data Catalog: таблиці + 62 партиції	те саме	$0 (free tier — перший 1 млн об’єктів)
Роль `AWSGlueServiceRole-ooad-lab8` + crawler	щоб можна було перезапустити Шлях Б	$0 у спокої

Три центи на місяць — ціна того, що результати лабораторної лишаються перевірюваними. Видалити все остаточно:

```
./teardown.sh --all
aws s3 rb s3://ooad-lab8-payments-<ACCOUNT_ID> --force
aws glue delete-crawler --name ooad-lab8-payments-crawler
aws iam delete-role-policy --role-name AWSGlueServiceRole-ooad-lab8 --policy-name read-ooad-lab8-payments-<ACCOUNT_ID>
aws iam detach-role-policy --role-name AWSGlueServiceRole-ooad-lab8 \
    --policy-arn arn:aws:iam::aws:policy/service-role/AWSGlueServiceRole
aws iam delete-role --role-name AWSGlueServiceRole-ooad-lab8
```